

Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introduction to Computer Architecture
Homework 10 (Due 12/8)

1. Consider a program with two threads running on the same **R10K-like processor**. Assume that each thread executes its four instructions over and over 100 times:

```
# thread A
A1: and  $t1, $t1, $t3
A2: lw   $t2, 0($t1)
A3: add  $t4, $t1, $t4
A4: sw   $t4, 4($t2)
# repeat 99 more times
```

```
# thread B
B1: ori  $s0, $s0, 1
B2: lw   $s3, 8($s0)
B3: sub  $s1, $s0, $s2
B4: sw   $s3, 0($s1)
# repeat 99 more times
```

Load/store instructions take four cycles in the X stage, add/sub instructions take two cycles, and all other instructions take one cycle. Assume that there are **infinitely** many reservation stations, functional units, physical registers and reorder buffer entries.

- a) You enable **fine-grained context-switching**. In the first cycle, the processor fetches an instruction from thread A, then in the next cycle, it fetches an instruction from thread B, and continues to alternate. Note that instructions are still retired in the order that they are fetched, even if they are from different threads.

Complete the last column (cycle 20) of the **pipeline diagram** on the next page for this program. How many **cycles** does it take to complete the entire program (from the F stage of the first instruction to the R stage of the last)?

Answer:

$$\begin{aligned}\text{Total cycles} &= (\# \text{ instructions}) + (\text{cycle latency per instruction} - 1) \\ &= (2 \times 4 \times 100) + (11 - 1) \\ &= 810\end{aligned}$$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
A1	F	De	Di	S	X	C	R													
B1		F	De	Di	S	X	C	R												
A2			F	De	Di	S	X	X	X	X	C	R								
B2				F	De	Di	S	X	X	X	X	C	R							
A3					F	De	Di	S	X	X	C	R	R	R						
B3						F	De	Di	S	X	X	C	R	R	R					
A4							F	De	Di	S	S	X	X	X	X	C	R			
B4								F	De	Di	S	S	X	X	X	X	C	R		
A1									F	De	Di	S	X	C	R	R	R	R	R	
B1										F	De	Di	S	X	C	R	R	R	R	R
A2											F	De	Di	S	X	X	X	X	X	R
B2												F	De	Di	S	X	X	X	X	C
A3													F	De	Di	S	X	X	C	R
B3														F	De	Di	S	X	X	C
A4															F	De	Di	S	S	X
B4																F	De	Di	S	S
A1																	F	De	Di	S
B1																		F	De	Di
A2																			F	De
B2																				F
A3																				
B3																				
A4																				
B4																				
A1																				
B1																				
A2																				
B2																				
A3																				
B3																				
A4																				
B4																				
A1																				
B1																				
A2																				
B2																				
A3																				
B3																				
A4																				
B4																				

b) You enable **two-wide fetch, decode, dispatch and retire** with **simultaneous multithreading**. In every cycle, the processor fetches one instruction each from threads

A and B. Up to two instructions can be retired at any cycle; though instructions are still retired in the order that they are fetched, even if they are from different threads.

Complete the last column (cycle 20) of the **pipeline diagram** on the next page for this program. How many **cycles** does it take to complete the entire program (from the F stage of the first instruction to the R stage of the last)?

Answer:

$$\begin{aligned}\text{Total cycles} &= (\# \text{ instructions}) / 2\text{-wide} + (\text{cycle latency per instruction} - 1) \\ &= (2 \times 4 \times 100) / 2 + (14 - 1) \\ &= 413\end{aligned}$$

- c) It turns out that enabling two-wide superscalar with simultaneous multithreading in Q1b increases the **clock period** by 85% compared to Q1a. Which of the two systems, Q1a or Q1b, yields a shorter total completion time of the program?

Answer:

$$(810 \text{ cycles} \times 1 \text{ clock period}) > (413 \text{ cycles} \times 1.85 \text{ clock period} = 764.05)$$

Therefore, Q1b yields lower runtime.

[illegible]

2. Consider four threads (T0, T1, T2, T3) running on a **quad-core multiprocessor** with private L1 data caches (C0, C1, C2, C3) and a single shared main memory. T0's cache is C0, T1's cache is C1, T2's is C2 and T3's is C3. The multiprocessor employs the **modified-shared-invalid (MSI) coherence protocol** from class for the private L1 data caches.

Below is a sequence of loads, stores and evicts issued by the threads to one **shared memory block X**. Initially block X is in the invalid (I) state in all four caches. For each load/store/evict in the sequence, specify the state of block X in each cache **after** performing the load/store/evict. The states at time 1 have been provided for you.

Time	Cache Access	State of Block X After Cache Access			
		C0	C1	C2	C3
0	initial state	I	I	I	I
1	T1 load X	I	S	I	I
2	T0 load X	S	S	I	I
3	T3 store X	I	I	I	M
4	T3 store X	I	I	I	M
5	T2 store X	I	I	M	I
6	T0 load X	S	I	S	I
7	T3 load X	S	I	S	S
8	T2 evict X	S	I	I	S
9	T0 store X	M	I	I	I
10	T1 load X	S	S	I	I
11	T1 store X	I	M	I	I
12	T1 load X	I	M	I	I
13	T2 store X	I	I	M	I
14	T3 load X	I	I	S	S
15	T3 evict X	I	I	S	I